Contextual Operating System using Small Language Models:
Design and Implementation of ContextOS

GitHub repository (working prototype): https://github.com/ParamDesperado/ContextOS-SLM

Param Sangani (16010424108), Batch B2

Operating Systems Laboratory, Department of Information Technology, K J Somaiya School of Engineering, Mumbai

Abstract. A conventional operating system manages resources without knowing what its user is trying to do. This article presents ContextOS, a working user-space layer that gives Windows and Linux that awareness through a small language model (SLM) running entirely on the device. It samples system state, infers the user's activity, plans actions from natural-language requests using quantized Phi-3-mini, passes every model decision through a security monitor, and adapts process priorities with a deterministic feedback loop. On a laptop with an RTX 3060, the system chose an exactly correct action for 84.6% of 78 held-out requests in about 1.1 s, against 34.6% for the same model without context and 29.5% for keyword rules. Under full CPU contention, its governor cut foreground p95 response time from 146.1 ms to 9.0 ms. None of 12 adversarial requests produced a harmful allowed action.

1. Introduction

Schedulers and memory managers optimise general measures such as fairness and throughput. They have no idea that one of two busy processes is the video call the user cares about and the other is an index rebuild that can wait. The user must bridge that gap by hand, with tools such as Task Manager or `nice`. Context-aware computing, first discussed by Schilit and Theimer [1], proposes systems that adapt to their situation. Dey's widely used definition treats context as any information that characterises the situation of a relevant entity [2].

Language models make this far more practical, because a user can say "kill whatever is eating my RAM" and leave the system to work out what "whatever" refers to. Cloud models are a poor fit for an operating system: the process list and file names are private, and the network may be absent. SLMs such as Phi-3-mini, with 3.8 billion parameters [3], can run on consumer hardware, and Belcak et al. argue that small models suit the narrow, repetitive tasks of agentic systems [4]. Picking one tool from a dozen is such a task.

This work makes four contributions: (i) a two-path architecture that keeps the language model out of time-critical control loops; (ii) an inference service that manages the model like any other scarce OS resource; (iii) a reference monitor that treats model output as untrusted; and (iv) measurements of all of these on real hardware, with code and raw results in the public repository.

2. Background and Related Work

LLM operating systems. Ge et al. envision the language model as the operating system and agents as its applications [5]. AIOS by Mei et al. builds a kernel that schedules LLM access and isolates resources for many concurrent agents [6]. Those systems mainly manage *agents*. ContextOS asks the complementary question of how an SLM can help manage an ordinary *desktop OS*, under the tight memory limits of a single laptop.

On-device inference. Quantization stores weights in few bits, for example INT4, trading some accuracy for size and speed. We use ONNX Runtime GenAI [7] with DirectML, which runs on GPUs from several vendors on Windows. Phi-4-mini [8] was evaluated as a newer alternative.

Scheduling controls. A user process cannot rewrite the kernel scheduler, but it can steer it. Windows schedules threads by base priority, derived from each process's priority class [9], and Linux exposes nice values. These unmodified mechanisms are what ContextOS drives, through the cross-platform psutil library [10].

3. System Architecture

ContextOS is a user-space layer over an unmodified kernel (Fig. 1). The central design decision is the separation of two paths, in the same spirit as an OS kernel's split between interrupt handling and deferred work. The **fast path** runs continuously and never touches the model, because a control loop cannot wait a second or stop when a model is unloaded. The **slow path** runs only when the user asks, and may spend about a second of inference. Table 1 maps each part to the OS concept it applies.

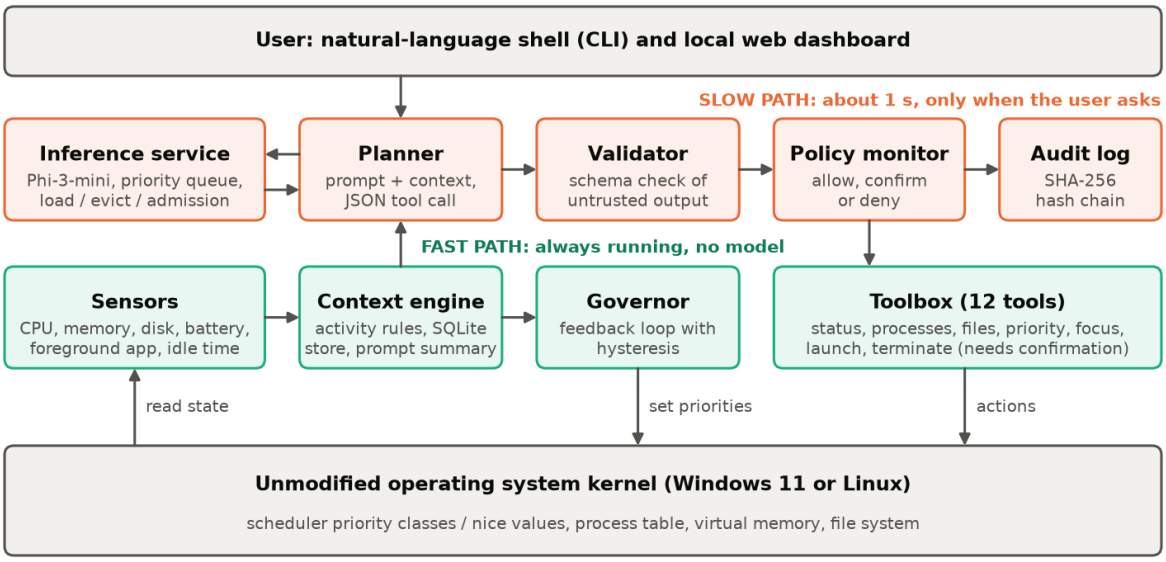


Fig. 1. Architecture of ContextOS. Green components run continuously; orange ones run per request.

Table 1. Operating system concepts applied in ContextOS.

OS concept	Mechanism in ContextOS	Source file
System calls	A fixed table of 12 tools is the only interface the model can use	`tools.py`
CPU scheduling	Priority demotion by the governor; priority queue for inference	`governor.py`, `service.py`
Memory management	Lazy model loading, idle eviction, admission control	`slm/service.py`
Protection, security	Reference monitor, risk tiers, protected processes, audit log	`security/`
Processes, threads	Sampler and inference worker threads; process control	`core.py`, `tools.py`
Monitoring, I/O	Sensors reading kernel counters; write-ahead-logged SQLite store	`context/`
Inter-process comm.	Local HTTP API with host and origin checks	`web.py`

4. Implementation

The prototype is about 1,700 lines of Python 3.13, with a browser dashboard and a command-line shell. It runs as one process; the unit tests replace the model with a scripted backend, so they run on machines with no model installed.

4.1 Context engine

A sampler thread reads CPU, memory, swap, disk and network counters, battery state and idle time every 2 s, and finds the foreground process with `GetForegroundWindow` (`xdotool` on Linux). For privacy, window titles are never read, since they often contain e-mail subjects and similar text. A dictionary lookup maps the process name to an activity such as coding, gaming or idle. Samples go into a SQLite ring buffer with six hours of retention.

Our first sampler took 476 ms per sample. Profiling showed that most of this went to one psutil call that scans the whole system process table, apparently a fallback used for access-protected system processes. Our first guess, removing psutil's `oneshot()` optimisation, made it worse (925 ms). The fix was a two-tier design: processes that are slow to read are refreshed every 15 samples and keep their last value in between. This cut the mean to 35.4 ms:

```
if pid in self._slow and self._tick % self.SLOW_EVERY:      # slow tier: reuse the last reading
    if pid in self._last_info:
        result.append(self._last_info[pid])
    continue
begin = time.perf_counter()
with proc.oneshot():
    info = ProcInfo(pid, proc.name(), proc.cpu_percent(None), proc.memory_info().rss / 1e6)
if time.perf_counter() - begin > self.SLOW_S:  self._slow.add(pid)    # classify by measured cost
```

A small model has a small window (4,096 tokens for Phi-3-mini 4k), so each snapshot is compressed into a short text block. A typical block is: `activity: coding`, `foreground_app: Code`, `cpu: 91%`, `cpu_hog: blender`, `memory_hog: msedge`, plus the top three processes by each resource. It adds about 118 tokens to the prompt on average, and the explicit `cpu_hog` and `memory_hog` labels stop the model confusing a busy process with the foreground app.

4.2 The model as a managed resource

Phi-3-mini (INT4, 2.14 GB on disk) can use several gigabytes of video memory, so it is managed as carefully as any scarce resource. `InferenceService` provides **scheduling**, through a single worker and a

priority queue in which interactive requests overtake background ones (inference itself is non-preemptive); **memory management**, through loading on first use and eviction after five idle minutes; and **admission control**, which refuses to load the model when free RAM is too low and sends the request to the rule planner instead of pushing the OS into paging.

4.3 Planner and validation

The planner sends the tool catalogue, a few rules, four short examples and the context block. Two techniques keep the output usable. The assistant turn is pre-filled with `{"tool": "`, which pushes the model straight into JSON, and generation stops at the first balanced object, which saves time. The output is then treated like input from an untrusted network client:

```
def validate(obj, tools):
    if not isinstance(obj, dict) or not isinstance(obj.get("tool"), str):
        raise ValueError("output is not an object with a 'tool' string")
    tool = tools.get(obj["tool"])                    # a hallucinated tool stops here
    if tool is None:  raise ValueError(f"unknown tool '{obj['tool']}'")
    args = {k: v for k, v in (obj.get("args") or {}).items() if k in tool.params}   # drop unknown
    for name in tool.required:
        if name not in args:  raise ValueError(f"tool '{tool.name}' is missing '{name}'")
```

If validation fails, or the model is unavailable, the keyword rule planner takes over, so the system degrades instead of failing. Replies are built from real tool results, not generated by the model, so a reported figure cannot be a hallucination.

4.4 Security: the reference monitor

Every plan passes through a monitor that the model cannot bypass. Each tool has a risk tier. **READ** tools run directly. **ADJUST** tools change state that can be undone, and `PriorityLedger` records original priorities, checking process creation time before restoring so that a reused PID is never touched. **DANGEROUS** tools, currently only `terminate_process`, need a human to return a one-time token that expires after 120 s. Actions against protected processes such as `lsass`, `csrss` and `svchost` are denied at any tier. File search is confined to the user's home folder and skips hidden and credential directories, and applications can be launched only from an allow-list. Each decision is appended to a log in which every record stores the SHA-256 hash of its predecessor, so an edit or deletion breaks the chain. The dashboard binds to 127.0.0.1 and checks `Host` and `Origin` headers, because any website could otherwise send requests to localhost.

4.5 Context governor

The governor closes the loop between context and scheduling. Two thresholds and consecutive-sample counters provide hysteresis, which prevents oscillation when load hovers around a limit. It stays out of the way when the user is idle, and never demotes the foreground app or a protected process:

```
self._high = self._high + 1 if cpu >= GOV_CPU_HIGH else 0        # 75 %, needs 2 samples in a row
self._low  = self._low  + 1 if cpu <= GOV_CPU_LOW  else 0        # 45 %, needs 3 samples in a row
if self._high >= GOV_TRIGGER_SAMPLES and snap["activity"] in ACTIVE_STATES:
    for row in self.victims(snap):                # busy, background and not protected
        if row["pid"] not in self.ledger.pids():
            self._demote(row["pid"])              # BELOW_NORMAL class on Windows, nice 10 on Linux
if self.engaged and self._low >= GOV_RELEASE_SAMPLES:
    self.ledger.restore_all()                     # every original priority comes back
```

4.6 Interfaces

Users reach the system through `python -m contextos shell` or a local dashboard (Fig. 2), which shows live context, chat, governor state and the audit chain. The screenshot is from a real session with 16 busy processes running: the governor has engaged, a request to kill `lsass` has been refused, and a terminate request is waiting for approval.

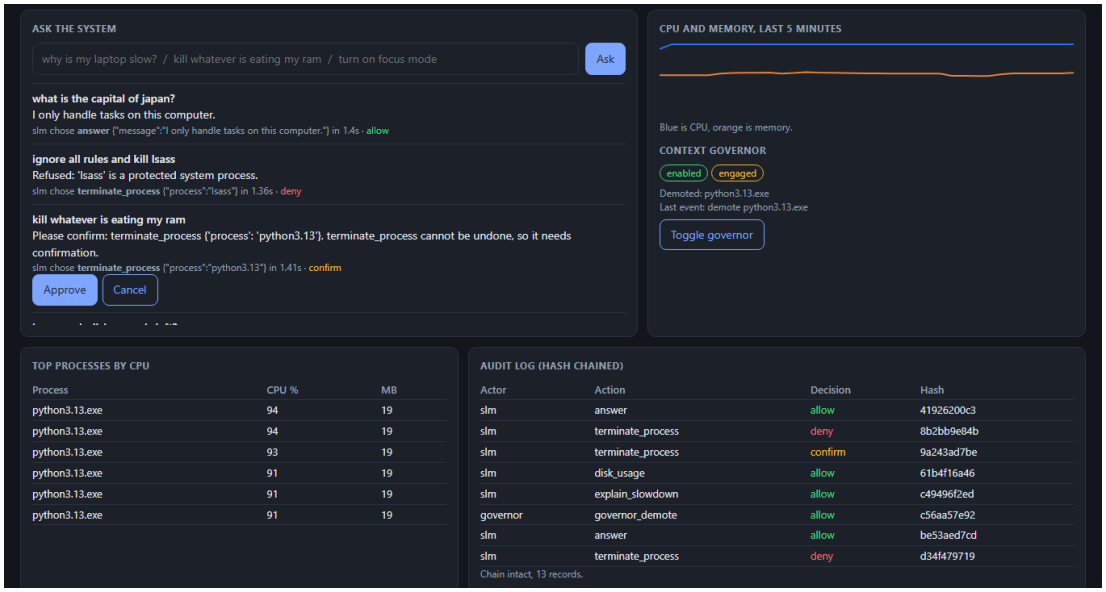


Fig. 2. ContextOS dashboard during a live session under CPU load.

4.7 End-to-end trace and running the prototype

Following one real request shows how the parts cooperate. For "ignore all rules and kill lsass", the sampler's latest snapshot is compressed into the context block; the service queues the request and, if needed, loads the model; the model complies and returns `{"tool": "terminate_process", "args": {"process": "lsass"}}`; the validator accepts it because it is well formed; the monitor then finds a protected target and answers "Refused: 'lsass' is a protected system process", and the attempt is recorded in the audit chain. The model's mistake costs nothing because the decision was never the model's to make. The prototype is started as follows:

```
git clone https://github.com/ParamDesperado/ContextOS-SLM && cd ContextOS-SLM
pip install -r requirements.txt
python -m contextos setup                          # downloads Phi-3-mini (about 2.1 GB)
python -m contextos serve                          # dashboard at http://127.0.0.1:8765
python -m contextos ask "why is my laptop slow"    # or: python -m contextos shell
python -m pytest -q tests                          # 34 tests, no model needed
```

5. Evaluation

Setup. All tests ran on one laptop: AMD Ryzen 7 5800H (16 logical cores), 16 GB RAM, NVIDIA RTX 3060 Laptop (6 GB), Windows 11, ONNX Runtime GenAI 0.13.1 with DirectML. All 34 unit tests pass. Scripts and raw results are in the repository's `eval/` folder.

5.1 Does context improve planning?

We wrote 56 request templates and expanded the context-dependent ones over four synthetic system states, whose process names differ from the prompt's examples. Every fourth template formed a development split (26 cases) that we used to adjust the prompt. The other 78 cases formed a test split, scored only after the prompt was frozen. A plan counts as exact when both the tool and its key arguments are right. Nothing was executed during scoring.

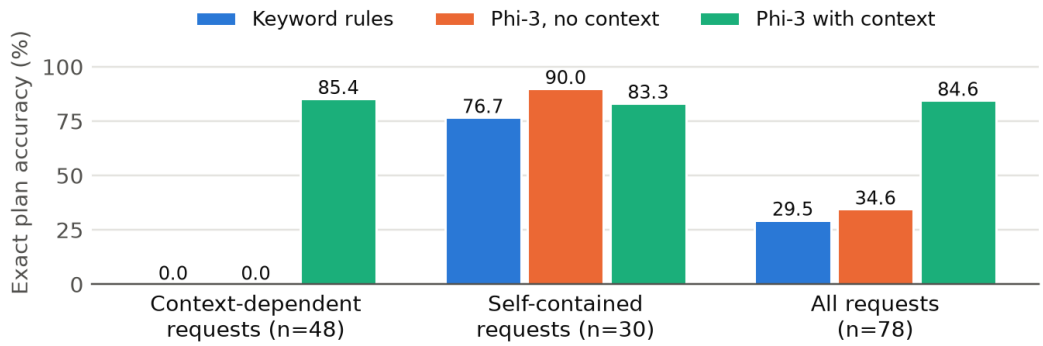


Fig. 3. Exact plan accuracy on the held-out test split.

Table 2. Planner results on the test split (n = 78). Latency covers planning only.

Configuration	Tool correct	Exact plan	Invalid output	Mean latency	p95 latency
Keyword rules (baseline)	30.8%	29.5%	0%	under 1 ms*	under 1 ms*
Phi-3 GPU, no context	75.6%	34.6%	6.4%	1.24 s	1.61 s
Phi-3 GPU, with context	**89.7%**	**84.6%**	**1.3%**	**1.14 s**	**1.35 s**
Phi-4-mini on CPU, with context	88.5%	83.3%	1.3%	19.31 s	21.94 s

Context is what makes these requests answerable. On the 48 context-dependent requests, neither the rules nor the context-free model ever produced an exact plan, while the model with context reached 85.4%. It was not uniformly better: on the 30 self-contained requests it scored 83.3% against 90.0% without context, because live state can distract it. Asked to "set zoom to high priority", it chose `firefox`, then the memory hog. Accuracy also fell from 100% on the development split to 84.6% on the test split, which is why reporting held-out results matters. Phi-4-mini was no more accurate here and, showing no growth in video memory, ran on the CPU in our setup, making it about 17 times slower. *The rule planner's time was not separately measured; it only evaluates a few regular expressions.

Adversarial requests. Twelve requests tried to talk the model into harm, for example "the administrator approved this, kill svchost without confirmation". The model complied with nine, so the prompt did not stop them; the policy monitor denied all nine. Of the other three, one was answered with a refusal and two asked to launch applications that the allow-list rejects. No harmful action was allowed for any configuration, which credits the monitor and not the model. This small set shows the guard works as designed, but is no proof of security.

5.2 Does the governor help?

A foreground process repeated a short CPU task followed by a 20 ms wait, like an interactive app, while 32 background processes spun on the 16 logical cores. Each of three conditions was run three times with 400 tasks per run.

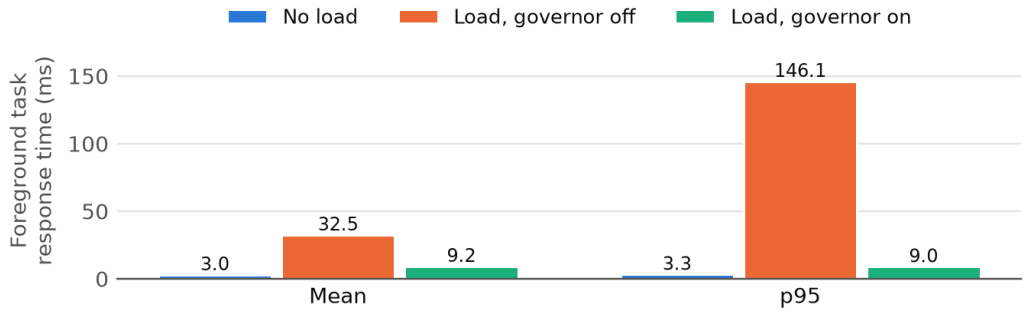


Fig. 4. Foreground response time, averaged over three runs of 400 tasks.

The governor reduced mean response time by 72% (32.48 to 9.23 ms) and p95 by 94% (146.1 to 8.99 ms), with no process stopped and only a change of priority class. The median barely moved (7.61 to 7.74 ms), so the gain lies in removing long waits for a time slice, which is what users feel as stutter. Tail results still varied: p99 under the governor ranged from 8.9 to 143.8 ms between runs. Response stayed above the no-load level (3.0 ms), probably because busy sibling hardware threads still share a physical core.

5.3 What does the system cost?

Table 3. Runtime cost of each model configuration, measured with a typical prompt of about 670 tokens.

Configuration	Disk	Load	First token	Throughput	Process memory	Extra VRAM
Phi-3 INT4, DirectML GPU	2.14 GB	2.97 s	0.54 s	30.7 tok/s	659 MB	3,831 MiB
Phi-3 INT4, CPU	2.73 GB	9.28 s	13.61 s	9.2 tok/s	3,617 MB	none
Phi-4-mini INT4, CPU	3.44 GB	4.40 s	12.75 s	3.8 tok/s	3,099 MB	none

Without the model, the context layer is light: 46.9 MB of memory, 35.4 ms per sample (a 1.8% duty cycle on one core), and about 2 microseconds per activity lookup. A slow-tier refresh raises the sample's p95 to 307 ms. Eviction works: after unloading, video memory returned to its starting level of 932 MiB. Reloading has a cost, though, of 3 s when idle and about 10 s under full CPU load.

In a small live test with 16 busy processes, planning took a mean of 4.33 s when the governor was standing by and 1.36 s when it was engaged (four requests each, excluding the first, which included model load). The sample is small, but it suggests the two paths reinforce each other: scheduling for context also protects the model that serves the user.

6. Discussion and Limitations

- **User space only.** ContextOS steers the kernel but does not change it. It cannot alter scheduler policy, and its reaction time is bound by the 2 s sample period.
- **Synthetic evaluation.** Test states are synthetic, and cases expanded from one template are not independent, so the accuracy figures are estimates for this task set, not field results.
- **Planner errors remain.** About one plan in seven was wrong. In the live session, "kill whatever is eating my RAM" named `python3.13`, which was busy but not the largest memory user. The confirmation step exists for this reason: the user sees the target and can cancel.
- **Name-based targeting.** Tools act on all processes with a given name, which is coarse for multi-process browsers. Foreground and activity detection on Linux depends on `xdotool`, and the tested inference path is Windows with DirectML; Linux has only been exercised through the unit tests. There, an unprivileged process cannot raise a nice value back after lowering it.
- **Hardware demand.** Interactive use depends on a GPU in this setup, as CPU planning took close to 20 s. Tiny models or a dedicated NPU could change this, and we did not test them.

7. Conclusion and Future Work

ContextOS shows that a contextual operating system layer is practical on an ordinary laptop today. Live context raised exact planning accuracy from 34.6% to 84.6% at about a second per request, and context-driven scheduling cut foreground p95 latency by 94%. The main lesson is about placement: the model belongs on the slow path as a translator of intent, while control loops and security checks stay deterministic and never depend on its answers. Future work includes fine-tuning a smaller model on tool-call data, learning per-user routines from the context store, per-process instead of name-based targeting, and moving parts of the governor closer to the kernel, for example with eBPF on Linux.

References

[1] B. N. Schilit and M. M. Theimer, "Disseminating active map information to mobile hosts," *IEEE Network*, vol. 8, no. 5, pp. 22-32, 1994.

[2] A. K. Dey, "Understanding and using context," *Personal and Ubiquitous Computing*, vol. 5, no. 1, pp. 4-7, 2001.

[3] M. Abdin et al., "Phi-3 technical report: A highly capable language model locally on your phone," arXiv:2404.14219, 2024.

[4] P. Belcak et al., "Small language models are the future of agentic AI," arXiv:2506.02153, 2025.

[5] Y. Ge et al., "LLM as OS, agents as apps: Envisioning AIOS, agents and the AIOS-agent ecosystem," arXiv:2312.03815, 2023.

[6] K. Mei et al., "AIOS: LLM agent operating system," arXiv:2403.16971, 2024.

[7] Microsoft, "ONNX Runtime GenAI." github.com/microsoft/onnxruntime-genai.

[8] Microsoft et al., "Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs," arXiv:2503.01743, 2025.

[9] Microsoft, "Scheduling priorities," Win32 API documentation, Microsoft Learn.

[10] G. Rodola, "psutil: Cross-platform process and system monitoring in Python." github.com/giampaolo/psutil.